Advanced Python Workshop & Task

Training

Workshop

Task 1

Prompt: Build a `BankAccount` class with a private balance (`_balance`), `__init__`, a `deposit()` and `withdraw()` method, and a read-only getter `balance` property. Withdrawals below zero should raise a `ValueError` instead of silently failing.

> **Check yourself:** Can someone do `account._balance = 999999` from outside and break it? (Discuss: Python doesn't have true private attributes — this is a convention, not a wall.)

> watch for: people using `@property` incorrectly, or skipping validation logic entirely.

Task 2

Prompt: Create a base `abstract` - *needs explain, not in session* - class `Shape` with an `area()` method that raises `NotImplementedError`. Create `Circle`, `Rectangle`, and `Triangle` subclasses that override it. Write a function `total_area(shapes: list)` that works on any mix of shapes without knowing their concrete type.
test type using `isinstance`; *explain why* `isinstance` *better than* `type`*?*

bonus: Add `__str__` so printing a shape gives a readable description - sets up Task 3.

Task 3

Prompt: Give the `Shape` classes from Task 2 support for `+`, `==`, and `<` (compare by area) using `__add__`, `__eq__`, `__lt__`. Also implement `__repr__` for debugging.

Check yourself: Does `sorted(list_of_shapes)` work without a `key=` argument? That confirms `__lt__` is correctly wired.

> forgetting `__eq__` also affects hashing — mention `__hash__` briefly if there is time.

Task 4

Prompt:

1. Add a class method `Shape.from_dict(data)` as an alternative constructor.
2. Add a static method (e.g., `Rectangle.is_square(width, height)`) that doesn't need `self` or `cls`. *explain why this it is a good idea*
3. Create an `Enum` class `Colour` (RED, GREEN, BLUE, WHITE) and add a `color` instance attribute default as WHITE and add a setter - *should check that the instance is of the enum class* - to change the colour as you wish. -> in Shape class to affect all subclasses at once

    > add it to the `__init__` in the Shape class allowing None as input, use `super.__init__()` at the begining of the subclasses for shared colour initialisation

> Could each method have correctly been a plain function instead? If a static method doesn't touch the class at all, ask why it's a method rather than a free function -good discussion prompt, no single right answer.

Online Task

Brief: Build a Small Library Management System

Design and implement a simplified library system that models books, members, and loans. Requirements:

1. **Class hierarchy:** an abstract base `LibraryItem` (use `abc.ABC` and/or `NotImplementedError`) with subclasses `Book`, `DVD`, and `Magazine` — each with different loan periods (polymorphism).
2. **Encapsulation:** each item tracks whether it's checked out; this state must not be directly settable from outside the class.
3. **Operator overloading:** items should be comparable/sortable by title (`__lt__`), and printable (`__repr__` / `__str__`). - *Alphabetical*
4. **Enums:** an `ItemStatus` enum (`AVAILABLE`, `CHECKED_OUT`, `LOST`) instead of magic strings.
5. **Class/static methods:** a class method to construct items from a dictionary (e.g., parsed from JSON), and a static method for a pure utility (e.g., validating an ISBN checksum).
6. **SOLID:** a `Library` class should manage the collection and checkouts, but should NOT be responsible for both persistence (saving/loading data) and business logic - split those into separate classes (SRP), and design it so adding a new item type doesn't require editing `Library`'s core logic (OCP).

> **Bonus**: apply singleton for the saving/loading data class